Universidad Nacional Autónoma de México.

Facultad de Ingeniería

"Sistemas Operativos"

Semestre: 2026-2

Grupo de Teoría: 07

Profesor:

DR. GUNNAR EYAL WOLF ISZAEVICH

– Trabajo Preliminar –

"Las GPU y los Sistemas Operativos"

Fecha de entrega: 16/03/2026

Equipo:

Adrián Axel Arzate Ríos

David Diaz Antunez

Índice

Índice **2**
Introducción **3**
Objetivos **3**
Desarrollo **4**
1. Relación entre el sistema operativo y la tarjeta gráfica 4
2. Contexto: kernel, librerías del sistema y system calls 4
3. DLL y bibliotecas compartidas 4
4. Wine y la compatibilidad con programas de Windows 5
5. DirectX y su importancia gráfica 6
6. OpenGL, Mantle, Vulkan y DXVK 7
Bibliografía **10**

Introducción

El sistema operativo y la tarjeta gráfica trabajan juntos para hacer posible la representación visual en una computadora. El sistema operativo administra recursos y coordina la comunicación entre programas y hardware, mientras que la tarjeta gráfica procesa y renderiza los gráficos que aparecen en pantalla.

Esta relación no ocurre de forma directa, sino a través de controladores, librerías del sistema y APIs gráficas. Gracias a estas capas, las aplicaciones pueden usar la GPU sin tener que comunicarse con el hardware de manera completamente manual.

En esta exposición se explicará cómo evolucionó esa relación y por qué fue clave para el desarrollo de videojuegos y aplicaciones gráficas.

Objetivos

- Explicar la relación entre el sistema operativo y la tarjeta gráfica.
- Entender, como contexto, el papel del kernel, las librerías del sistema y las system calls.
- Comprender la importancia de las syscall y su relación con la compatibilidad de software.
- Analizar cómo Wine permitió ejecutar software de Windows en Linux.
- Explicar el papel de DirectX en el desarrollo gráfico moderno.
- Explorar la compatibilidad de aplicaciones binarias con uso de gráficos desarrolladas para Windows en entornos Linux.

Desarrollo

1. Relación entre el sistema operativo y la tarjeta gráfica

La GPU no trabaja sola: necesita que el sistema operativo organice su uso mediante controladores y APIs. Por sus siglas en inglés Application Programming Interface

Cuando una aplicación quiere dibujar algo en pantalla, normalmente no le habla directamente a la tarjeta gráfica. En su lugar, usa una API gráfica, que se apoya en librerías, drivers y servicios del sistema operativo para enviar instrucciones a la GPU.

La idea central es esta:

Aplicación → API gráfica → driver → sistema operativo → GPU

2. Contexto: kernel, librerías del sistema y system calls

Como contexto, el sistema operativo actúa como intermediario entre los programas y el hardware. Su parte central es el kernel, que administra CPU, memoria, procesos y dispositivos.

Las aplicaciones normalmente no acceden al hardware directamente. En cambio, usan librerías del sistema, y estas pueden hacer system calls, que son solicitudes formales al kernel.

3. DLL y bibliotecas compartidas

A medida que el software se volvió más complejo, surgieron las bibliotecas compartidas. En Windows, una de sus formas más conocidas son las DLL.

Una DLL es una biblioteca dinámica que contiene funciones reutilizables por distintos programas. No es un “gestor”, sino un componente que permite modularidad y reutilización de código. Esto fue importante porque muchas funciones del sistema y muchas APIs de Windows se distribuyen precisamente mediante DLL.

4. Wine y la compatibilidad con programas de Windows

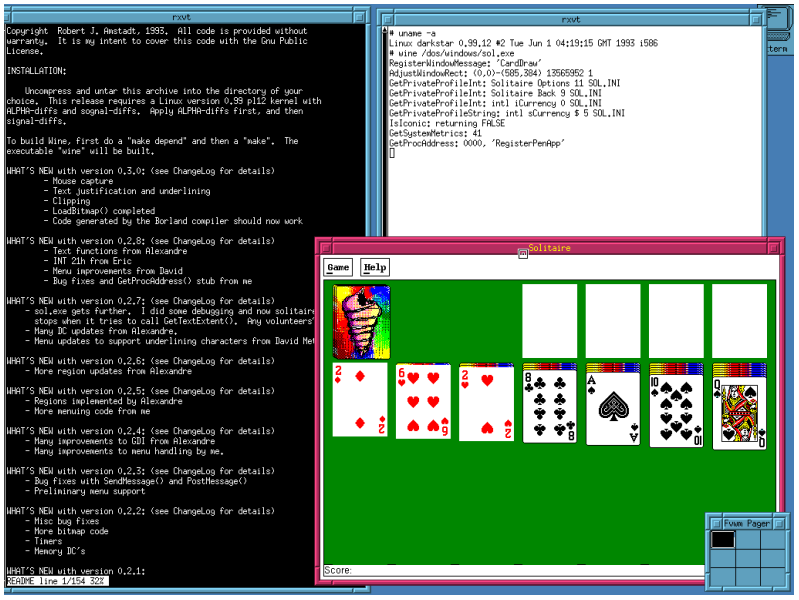
Wine fue una pieza importante porque muchos juegos y programas gráficos fueron desarrollados originalmente para Windows y, por lo tanto, esperaban encontrar su mismo entorno de ejecución: ciertas DLL, funciones del sistema y APIs gráficas como DirectX.

Esto lo vuelve relevante porque permite entender que un programa no funciona solo por estar “instalado”, sino porque depende de varias capas de software. Por ejemplo, muchas aplicaciones de Windows llaman funciones que están dentro de DLL; esas funciones, a su vez, pueden apoyarse en servicios del sistema operativo y en APIs especializadas para gráficos. Cuando ese mismo programa se intenta ejecutar en Linux, ese entorno no existe de forma nativa.

Ahí es donde aparece Wine. Su importancia está en que actúa como una capa de compatibilidad capaz de reproducir o traducir muchas de las funciones que un programa de Windows espera encontrar. Gracias a esto, Wine ayuda a mostrar con claridad cómo se relacionan varios conceptos:

- las DLL, como bibliotecas que contienen funciones reutilizables.
- las system calls, como mecanismo mediante el cual el sistema operativo ofrece servicios.
- y las APIs gráficas, como DirectX, que permiten a los programas comunicarse con la GPU de forma estandarizada.

Por eso, Wine no solo fue útil para ejecutar programas de Windows en Linux, sino también para evidenciar que la compatibilidad entre sistemas no depende únicamente del hardware, sino de todo un ecosistema de bibliotecas, interfaces y mecanismos de comunicación entre el software y el sistema operativo.



https://imgur.com/gallery/wine-0-3-0-from-september-1993-slackware-1-01-D82U961

5. DirectX y su importancia gráfica

A comienzos de los años 2000, programar para GPU era más complicado porque existían distintas marcas de tarjetas gráficas y muchas diferencias entre sus capacidades y controladores. Esto hacía que desarrollar software gráfico y videojuegos fuera más difícil, ya que no siempre había una forma uniforme de aprovechar el hardware.

En ese contexto, DirectX, y en particular Direct3D, se volvió muy importante porque ofreció una interfaz común para trabajar con aceleración gráfica en Windows. Gracias a ello, los desarrolladores ya no tenían que pensar tanto en cada tarjeta por separado, sino en una API estandarizada dentro del ecosistema de Microsoft. Eso facilitó mucho el desarrollo de videojuegos y ayudó a consolidar a Windows como la plataforma dominante para gráficos en PC.

Sin embargo, esa misma ventaja también generó una fuerte dependencia del ecosistema de Microsoft. Muchos programas y juegos quedaron diseñados alrededor de DirectX y de las bibliotecas de Windows, lo que dificultó su ejecución en otros sistemas como Linux. En ese sentido, DirectX funcionó no solo como un estándar técnico, sino también como un factor de concentración del desarrollo gráfico en Windows.

Esta dificultad con el ecosistema gráfico de Linux se reflejó incluso en críticas muy conocidas dentro de la comunidad. En 2012, Linus Torvalds calificó a NVIDIA como “the single worst company we’ve ever dealt with” y remató con su famoso “fuck you” en referencia a los problemas de soporte para Linux, especialmente en un contexto de frustración por la falta de apertura y colaboración con la plataforma.

Linus Torvalds: Nvidia, Fuck You!

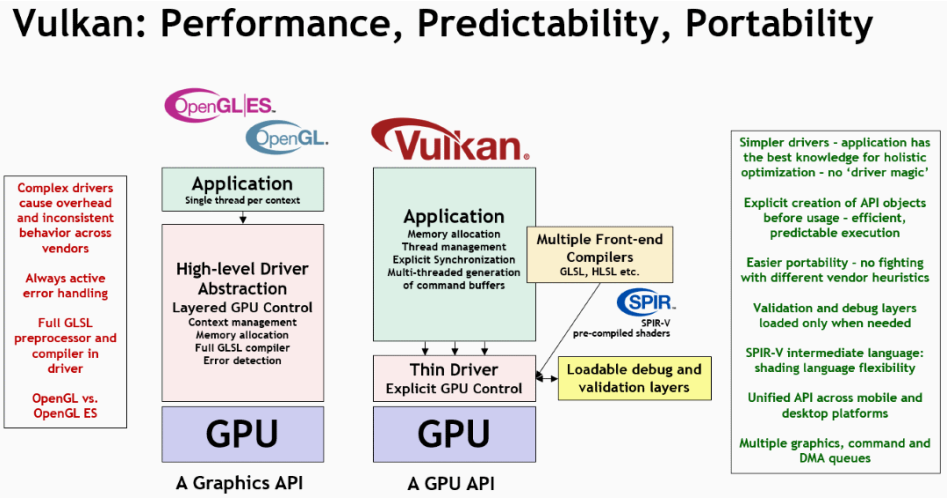
6. OpenGL, Mantle, Vulkan y DXVK

Aunque DirectX se convirtió en la referencia dominante para el desarrollo gráfico en Windows, no fue la única forma de comunicarse con la GPU. Con el tiempo surgieron distintas alternativas que buscaron resolver el mismo problema desde enfoques diferentes, especialmente fuera del ecosistema de Microsoft.

Una de las más importantes fue OpenGL, una API gráfica multiplataforma que permitió desarrollar aplicaciones con mayor portabilidad entre sistemas operativos. Su valor estuvo en ofrecer una alternativa fuera del entorno exclusivo de Windows, algo especialmente importante para Linux.

Después apareció Mantle, desarrollada por AMD, con la idea de reducir la sobrecarga entre software y hardware y dar un acceso más directo a la GPU. Aunque Mantle no se convirtió en el estándar dominante, sí fue importante porque impulsó la idea de APIs de más bajo nivel y más cercanas al hardware.

Esa misma dirección se consolidó con Vulkan, mantenida por Khronos. Vulkan está pensada para dar un acceso eficiente, multiplataforma y de bajo nivel a GPUs modernas, y puede entenderse como una evolución del enfoque que buscaban APIs como Mantle. Khronos la define como una API de gráficos y cómputo de nueva generación para programar GPUs modernas de forma multiplataforma y entre distintos fabricantes.



Más adelante, en el contexto de Linux y la compatibilidad con juegos de Windows, apareció DXVK. DXVK es una capa de traducción basada en Vulkan que implementa Direct3D 8, 9, 10 y 11 sobre Vulkan, permitiendo ejecutar muchas aplicaciones 3D de Windows en Linux mediante Wine. Gracias a esto, Vulkan no solo fue importante como API propia, sino también como base para mejorar la compatibilidad del ecosistema gráfico de Linux con juegos pensados para DirectX.

En conjunto, estas tecnologías muestran que la relación entre sistema operativo y GPU no depende de una sola API. DirectX dominó Windows, OpenGL ofreció portabilidad, Mantle

impulsó el enfoque de bajo nivel, Vulkan consolidó esa idea de forma multiplataforma y DXVK permitió traducir gran parte del mundo de DirectX al ecosistema Linux.

Parte técnica

Microsoft documenta qué debe hacer la API; Wine reconstruye una implementación compatible para que ese mismo programa funcione en Linux. https://learn.microsoft.com/es-es/windows/win32/api/_directml/ (documentación de las .dll de Windows).

Wine hace visible su trabajo cuando un programa de Windows intenta usar una API gráfica que Linux no tiene de forma nativa. En ese momento, Wine debe ofrecer una implementación compatible de las DLL y del comportamiento que la aplicación espera. En gráficos, eso significa reproducir llamadas de Direct3D y apoyarse en capas como DXVK para traducirlas al stack gráfico de Linux.

Un ejemplo real del código de pruebas de Wine:

```
hr = IDirect3DDevice9Ex_BeginScene(device);
ok(SUCCEEDED(hr), "Failed to begin scene, hr %#lx.\n", hr);

hr = IDirect3DDevice9Ex_DrawPrimitiveUP(device, D3DPT_TRIANGLESTRIP, 2, quad, sizeof(quad[0]));
ok(SUCCEEDED(hr), "Failed to draw, hr %#lx.\n", hr);

hr = IDirect3DDevice9Ex_EndScene(device);
ok(SUCCEEDED(hr), "Failed to end scene, hr %#lx.\n", hr);

hr = IDirect3DDevice9Ex_Present(device, NULL, NULL, NULL, NULL);
ok(SUCCEEDED(hr), "Failed to present, hr %#lx.\n", hr);
```

Este fragmento muestra una prueba de Wine sobre funciones de Direct3D 9. Su objetivo es comprobar que las llamadas gráficas se comporten de forma correcta dentro de la capa de compatibilidad.

Primero, ***BeginScene(device)*** indica el inicio de una escena de renderizado. En otras palabras, le dice al sistema gráfico que va a comenzar el proceso de dibujar objetos en pantalla.

Después, ***DrawPrimitiveUP(args)*** dibuja una figura usando los datos que recibe directamente en memoria. En este caso, la función intenta renderizar primitivas gráficas, es decir, formas básicas con las que se construyen objetos más complejos.

Luego, ***EndScene(device)*** marca el final de la escena. Esto significa que ya terminaron las instrucciones principales de dibujo para ese cuadro.

Por último, ***Present(args)*** presenta el resultado en pantalla. Esta es la parte donde la imagen generada se envía al búfer de salida para que pueda mostrarse visualmente.

En cada paso aparece la macro ***ok(args).*** Esa macro se usa en las pruebas de Wine para verificar si la llamada funcionó correctamente. Si el resultado no es exitoso, imprime un mensaje de error indicando en qué parte falló.
La importancia de este fragmento es que muestra de forma tangible qué hace Wine: no solo carga un programa de Windows, sino que también debe reproducir correctamente el comportamiento de sus APIs gráficas. Si estas llamadas no responden como lo harían en Windows, el programa o videojuego no renderizaría bien la imagen, o simplemente fallaría.

El siguiente diagrama muestra cómo una aplicación de Windows llega a usar la GPU en Linux:

Aplicación
↓
API gráfica
(DirectX / DirectML)
↓
Capa de compatibilidad
(Wine)
↓
Capa de traducción gráfica
(DXVK o vkd3d-proton)
↓
Driver / Vulkan
↓
Sistema operativo Linux
↓
GPU

La aplicación envía instrucciones gráficas mediante una API como DirectX o DirectML.
Esas llamadas pasan por Wine, que actúa como capa de compatibilidad y reproduce el entorno que el programa espera encontrar en Windows.

Después entra una capa de traducción gráfica, como DXVK o vkd3d-proton, que convierte esas llamadas a un formato que Linux sí puede usar, normalmente Vulkan.

Luego intervienen el driver y el sistema operativo Linux, que coordinan el acceso al hardware.

Finalmente, las instrucciones llegan a la GPU, que procesa y muestra la imagen.

Bibliografía

- WineHQ. (s. f.). *d3d9ex.c*. GitHub. Recuperado el martes, 11 de marzo de 2026, de https://github.com/wine-mirror/wine/blob/master/dlls/d3d9/tests/d3d9ex.c
- Werat. (2022, 10 de octubre). *How Wine works 101*. Recuperado el miércoles, 12 de marzo de 2026, de https://werat.dev/blog/how-wine-works-101/
- YouTube. (s. f.). *[Video]*. Recuperado el martes, 11 de marzo de 2026, de https://www.youtube.com/watch?v=eqPQc0Y5HKE&t=62s
- Khronos Group. (s. f.). *What is Vulkan?* Recuperado el miércoles, 12 de marzo de 2026, de https://github.khronos.org/Vulkan-Site/guide/latest/what_is_vulkan.html
- Profesional Review. (2019, 14 de mayo). *AMD fin API Mantle*. Recuperado el martes, 11 de marzo de 2026, de https://www.profesionalreview.com/2019/05/14/amd-fin-api-mantle/
- Imgur. (s. f.). *Wine 0.3.0 from September 1993 Slackware 1.01* [Imagen]. Recuperado el miércoles, 12 de marzo de 2026, de https://imgur.com/gallery/wine-0-3-0-from-september-1993-slackware-1-01-D82U961
- YouTube. (s. f.). *[Video]*. Recuperado el martes, 11 de marzo de 2026, de https://www.youtube.com/watch?v=4daUujBgQQ8